Entrega de Tarea 2

1. **Aplicación o mini-juego funcional:** Mini-juego “Radar93”
2. **Evidencia de videcoding (prompts)**

Primer Prompt

- Eres un ingeniero en informática especializado en diseño, creación e implementación de videojuegos. Tu tarea es desarrollar un videojuego llamado Radar93, inspirado en Flappy Bird.
- Concepto del juego
- El jugador controla un avión comercial que debe volar a través de un espacio lleno de rascacielos estilo Nueva York, en lugar de las tuberías de Flappy Bird.
- El avión avanza automáticamente y el jugador controla su altura para evitar chocar con los edificios.
- Los edificios son rectangulares, con ventanas iluminadas al estilo Nueva York, siluetas detalladas y antenas. Usa formas simples de colores sólidos y un estilo minimalista.
- El fondo representa un atardecer.
- Cuando el avión choca con un edificio, el avión se estrella y cae. El edificio produce únicamente un poco de fuego y humo como efecto visual.
- Niveles
- Nivel Histórico: velocidad normal.
- Nivel Terrorífico: un poco más rápido que el Histórico.
- Solo existen estos dos niveles. El jugador debe poder elegir entre ellos antes de comenzar a jugar.
- Sonido y efectos
- Incluye sonidos y efectos solo si no requieren recursos externos ni complican significativamente la implementación. Si no es posible sin complicaciones, omítelos y repórtalo en tu plan.
- Restricciones técnicas
- Desarrolla el juego únicamente con Python y su librería estándar, usando pygame como librería gráfica. Antes de empezar la implementación, confirma conmigo que estoy de acuerdo con el uso de pygame. No se permiten otros frameworks externos.
- Versión de Python: 3.15.0. No uses otra versión como referencia.
- El entregable debe ser un único archivo .py que se pueda ejecutar directamente desde VS Code.
- Calidad del código
- Aplica buenas prácticas SOLID y patrones de diseño cuando realmente aporten valor (por ejemplo, separar la lógica del juego, el renderizado y la gestión de niveles).
- No incluyas comentarios en cada línea. En cambio, coloca un comentario al inicio de cada función indicando de forma concreta qué hace.
- Alcance y restricciones de trabajo

- No agregues funcionalidades, mecánicas, niveles, menús o elementos adicionales sin mi autorización explícita.
- Si tienes alguna sugerencia de mejora o de implementación, indícamela antes de comenzar el desarrollo para que yo decida si la aceptamos.
- Antes de escribir el código, expón brevemente tu plan de arquitectura, las sugerencias que tengas (incluida la confirmación del uso de pygame) y espera mi confirmación para proceder con la implementación completa.

Explicación: Se pidió la esencia de lo que yo quería como mini juego, asemejando al juego Flappy Bird, la cual sería la base para ir retocando más adelante el mini juego, en ese sentido se incluyó ciertas modificaciones, tales como formato del juego, sus niveles, su diseño, sonidos y efectos. También se pidió formato y lo que se esperaba como respuesta al código del juego, aplicando buenas prácticas. Se refinó el prompt con prompt cowboy, como se enseñó en clase, dando la estructura de un buen prompt: su contexto, la instrucción, su formato y las restricciones.

Segundo Prompt

Eres un desarrollador senior de videojuegos en Python con experiencia en pygame y pygame-ce. Tu tarea es escribir un juego de arcade de esquivar obstáculos con avión, completo y ejecutable en un solo archivo, siguiendo exactamente los requisitos que describo a continuación.

Plataforma y dependencias

- El juego debe funcionar con pip install pygame. Usa pygame-ce solo si la instalación de pygame no tiene ruedas (wheels) disponibles para el sistema del usuario: implementa una importación con respaldo, por ejemplo try: import pygame y, si falla, instrucciones claras para instalar pygame-ce. Mantén el mismo código para ambas variantes.
- La resolución de la ventana debe ser exactamente 800x600 píxeles.

Controles

- Solo se usan dos teclas para volar: la flecha arriba y la tecla "W". Ninguna otra tecla debe tener efecto en el juego, y los mensajes del juego no deben mencionar ni depender de otras teclas.
- Al inicio de la partida (antes de empezar a jugar), muestra un mensaje breve en pantalla que indique que solo las teclas flecha arriba y W sirven para volar.

Tono y mensajes

- El tono del juego es retro y directo, con estilo arcade: mensajes cortos, sin texto de relleno.

Sonido

- Genera los efectos de sonido por código (por ejemplo, con pygame.sndarray o construyendo ondas sinusoidales o de ruido), sin depender de archivos externos.
- Si el sistema de audio no está disponible o falla la inicialización del mixer, el juego debe seguir funcionando sin sonido, sin lanzar errores.

Puntuación

- Muestra en la esquina superior derecha de la pantalla el tiempo de duración de la partida actual.
- La puntuación se calcula según el tiempo sobrevivido, de modo que al terminar la partida el resultado final sea ese tiempo.

Flujo del juego

1. Al iniciar, el usuario ve el menú principal, donde puede seleccionar el nivel a jugar.
2. Al seleccionar un nivel, comienza la partida, con la pantalla de instrucciones de controles mostrada al inicio.
3. Cuando el avión choca con un obstáculo, aparece una pantalla que dice "Perdiste" con un mensaje de ánimo breve tipo "Más suerte la próxima". Esta pantalla espera a que el usuario presione ENTER.
4. Al presionar ENTER, el usuario regresa al menú principal para elegir otro nivel.

Entregable

- Un único archivo Python ejecutable, con todo el código comentado en español donde sea útil, sin dependencias de recursos externos (imágenes y sonidos se generan o se dibujan en código).
- Al final, incluye las instrucciones exactas para ejecutarlo: instalar con pip install pygame y correr con python nombre_del_archivo.py, además de la nota de cómo instalar pygame-ce si el usuario lo necesita.

Explicación: Luego de una pequeña revisión de lo pedido, se ordenó ajustar el tamaño de la pantalla, que implementara una importación de respaldo en caso el pygame no funcionara con la versión con la que tengo Python en mi máquina. De la misma manera, se indicó cuáles serían los controles del juego y el flujo del funcionamiento del juego como lo es la puntuación y el tono del juego, finalizando con la inclusión de tonos y efectos para hacer más divertido al juego. Siempre se ajustó el prompt siguiendo la estructura de un buen promp y de acuerdo con las recomendaciones de prompt cowboy, con el fin de transformar el lazy prompt inicial.

Tercer Prompt

Eres un desarrollador de videojuegos senior con experiencia en Python y pygame. Tu tarea es modificar un juego existente tipo Flappy Bird con avión, escrito en pygame, aplicando los cambios concretos que se detallan abajo. Escribe el código completo y funcional de los archivos modificados, indicando claramente qué archivo reemplaza a cuál.

Estado actual que debes respetar: el juego ya funciona en pygame, incluyendo sonido y efectos. No cambies la lógica que funciona: solo modifica lo indicado. Mantén sin cambios los efectos de sonido del humo y la explosión.

Cambios requeridos:

1. Obstáculos en ambos lados: actualmente los edificios siempre están en la parte inferior, por lo que el avión que vuela arriba es siempre seguro. Haz que los obstáculos aparezcan también desde arriba, estilo Flappy Bird: pares de obstáculos superior e inferior con un hueco por el que debe pasar el avión. La posición vertical del hueco debe variar entre partidas y dentro del juego, y debe situarse en una zona que obligue al jugador a moverse arriba y abajo. Los edificios de la parte superior también deben funcionar como obstáculos, con variedad de altura.

2. Diferencia visible entre niveles: los dos niveles actualmente no se distinguen. Haz que el nivel 2 sea claramente más difícil y se note al jugar, mediante velocidad de desplazamiento y separación de obstáculos distintas en cada nivel, y con un fondo o paleta de color diferente para cada nivel. Puedes complementarlo con huecos más estrechos o mayor frecuencia de obstáculos en el nivel 2. Muestra el número de nivel en pantalla al empezar cada uno.

3. Humo y explosión más dramáticos: aumenta el tamaño, la cantidad de partículas, el alcance y la duración del humo y de la explosión, de forma que se vea más dramático y se noten claramente los cambios. Mantén sus efectos de sonido sin cambios.

4. Atardecer más rojizo y púrpura, de tarde-noche: ajusta el degradado del cielo hacia tonos rojo-violeta intensos en el horizonte y azul-púrpura oscuro hacia arriba, de modo que se perciba como un atardecer avanzado que se acerca a la noche.

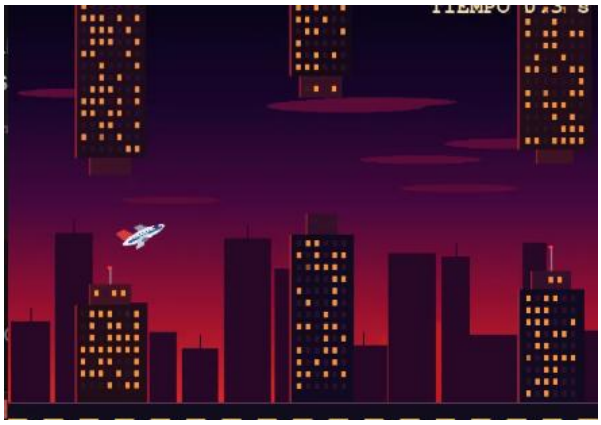
5. Luces de edificios que parpadean: haz que algunas ventanas iluminadas de los edificios se enciendan y se apaguen durante el juego, de forma aleatoria e independiente, con transiciones simples. No todas deben cambiar a la vez.

Entrega: devuelve el código Python completo de cada archivo modificado, con comentarios breves en español señalando dónde se implementa cada cambio, y sin alterar el sonido ni las partes del juego que no se mencionan.

Explicación: Luego de probar y revisar, se indicó hacer unas modificaciones con respecto a la dinámica, dificultad y estética del juego, señalando mejoras rápidas referido a los colores del fondo, la ejecución de luces parpadeantes durante la marcha del juego, la aplicación de obstáculos tanto arriba como abajo del mapa, la velocidad de ambos niveles y mejora en los efectos de choque. De la misma manera, se siguió los pasos y estructura de un buen prompt ocupando prompt cowboy.

3. **Iteración y mejora (mejora visual)**





4. **Validación del resultado**

Explicación de prueba del código: Como ya se tenía la mayor parte de las cosas instaladas, solo se instaló pygame a través del cmd de la máquina ocupando pip install pygame. Posteriormente se ubicó por medio del cmd a la carpeta en la que se estaba trabajando y se ejecutó el comando python 93.py, la cual corresponde al nombre del archivo, para así poderlo correr y hacer las respectivas pruebas dentro del juego, haciendo comparaciones y testing de lo que se le pidió a la IA. Por consiguiente, luego de varias partidas de prueba, se confirmó que el juego cumplía con todo lo que se había pedido a través de los prompts.

Limitación: Se hicieron varios cambios a través de los prompts, sin embargo, el que más me llamó la atención fue que a pesar de que se había indicado que fuera como flappy bird, donde tiene obstáculos como arriba y abajo (solo dejando espacio en medio para que pase el pájaro), la IA en una primera versión solo había colocado obstáculos abajo, siguiendo el funcionamiento de una ciudad normal, los edificios nada más abajo. Por lo que se tuvo que hacer un énfasis en dicho cambio, para poder cumplir con el objetivo inicial del juego.

5. **Reflexión final**

Qué aprendí usando IA para programar

Pues aprendí que el resultado de lo que da la IA depende mucho de cómo describo y escribo lo que quiero como resultado. Cuando fui concreto diciendo que solo quería un archivo, solo pygame, solo dos niveles, solo ciertas teclas, etc. el código salió casi como lo pedí. Asimismo, vi que pedir un plan antes de programar evita malentendidos, y que trabajar por versiones funciona bien oucpando few shots, de modo que, primero tuve un juego básico y después fui modificando agregando cambios al juego, tomando decisiones a partir de lo que daba como resultado la IA.

Ventajas y límites del vibecoding

La mayor ventaja que posee es la velocidad, ya que me sorprendió que con un tan solo un prompt ya tenía más de la mitad del minijuego hecho, ya que en poco tiempo obtuve un juego completo con sonido generado por código, partículas y estados de juego, algo que si no fuera por la IA me hubiera tardado un mundo escribiendo e investigando de cómo hacerlo y me quedara como yo quisiese. También, sirvió para explorar ideas para la organización y trabajo de los niveles de juego.

Dentro de los límites hay varios, por ejemplo la IA no puede oír el sonido ni sentir si el juego es difícil, sino que son los números de velocidad y de huecos, los que había que establecerse para hacer esas estimaciones de dificultad y que solo pueden notarse jugando.

Qué comprendo y qué debo reforzar

Lo que entiendo sería el bucle principal del juego, las constantes y la configuración de niveles en LEVELS, los estados del juego como menú, partida, derrota y las colisiones con rectángulos.

Lo que debo reforzar sería:

- La generación de sonido con array y ondas senoidales
- La transparencia en las partículas de fuego y humo
- El algoritmo que elige la altura del siguiente hueco
- El parpadeo de ventanas con temporizadores y transiciones
- El uso de dataclass y de la clase State